

```

1 !pip install -q -U bitsandbytes
2 !pip install -q -U git+https://github.com/huggingface/transformers.git
3 !pip install -q -U git+https://github.com/huggingface/peft.git
4 !pip install -q -U git+https://github.com/huggingface/accelerate.git
5 !pip install -q datasets

```

```

----- 122.4/122.4 MB 5.3 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
----- 3.0/3.0 MB 37.5 MB/s eta 0:00:00
Building wheel for transformers (pyproject.toml) ... done
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheel for peft (pyproject.toml) ... done
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheel for accelerate (pyproject.toml) ... done
----- 471.6/471.6 kB 15.3 MB/s eta 0:00:00
----- 116.3/116.3 kB 9.5 MB/s eta 0:00:00
----- 134.8/134.8 kB 11.3 MB/s eta 0:00:00
----- 194.1/194.1 kB 14.8 MB/s eta 0:00:00

```

First let's load the model we are going to use - GPT-neo-x-20B! Note that the model itself is around 40GB in half precision

```

1 import os
2 os.environ['CUDA_LAUNCH_BLOCKING'] = '1'
3

```

```

1 import torch
2 print(torch.cuda.is_available()) # Should return True if GPU is enabled
3

```

True

```

1 Start coding or generate with AI.

```

```

1 from transformers import TrainingArguments, trainer

```

```

1 import torch
2 from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
3
4 model_id = "EleutherAI/gpt-neo-1.3B"
5 bnb_config = BitsAndBytesConfig(
6     load_in_4bit=True,
7     bnb_4bit_use_double_quant=True,
8     bnb_4bit_quant_type="nf4",
9     bnb_4bit_compute_dtype=torch.bfloat16
10 )
11
12 tokenizer = AutoTokenizer.from_pretrained(model_id)
13 model = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb_config, device_map="auto")

```

/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```

warnings.warn(
tokenizer_config.json: 100% 200/200 [00:00<00:00, 12.0kB/s]
config.json: 100% 1.35k/1.35k [00:00<00:00, 76.9kB/s]
vocab.json: 100% 798k/798k [00:00<00:00, 6.19MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 2.01MB/s]
special_tokens_map.json: 100% 90.0/90.0 [00:00<00:00, 5.40kB/s]
model.safetensors: 100% 5.31G/5.31G [00:30<00:00, 115MB/s]

```

Then we have to apply some preprocessing to the model to prepare it for training. For that use the

`prepare_model_for_kbit_training` method from PEFT.

```

1 # from peft import prepare_model_for_kbit_training
2
3 # # model.gradient_checkpointing_enable()
4
5
6 # model = prepare_model_for_kbit_training(model)

```

```

1 def print_trainable_parameters(model):
2     trainable_params = 0
3     all_param = 0
4     for _, param in model.named_parameters():
5         all_param += param.numel()
6         if param.requires_grad:
7             trainable_params += param.numel()
8     print(
9         f"trainable params: {trainable_params} || all params: {all_param} || trainable%: {100 * trainable_params / all_
10     )

```

```

1 from peft import LoraConfig, get_peft_model
2
3 config = LoraConfig(
4     r=8,
5     lora_alpha=32,
6     # target_modules=['query', 'key', 'value', 'dense'],
7     lora_dropout=0.05,
8     bias="none",
9     task_type="CAUSAL_LM"
10 )
11
12
13 model = get_peft_model(model, config)
14 print_trainable_parameters(model)

```

trainable params: 1572864 || all params: 713168896 || trainable%: 0.2205457934048767

Let's load a common dataset, english quotes, to fine tune our model on famous quotes.

```

1 from datasets import load_dataset
2
3 dataset = load_dataset("neulab/conala")
4

```

conala.py: 100% 4.30k/4.30k [00:00<00:00, 253kB/s]

README.md: 100% 3.90k/3.90k [00:00<00:00, 226kB/s]

The repository for neulab/conala contains custom code which must be executed to correctly load the dataset. You can inspect You can avoid this prompt in future by passing the argument `trust\_remote\_code=True`.

Do you wish to run the custom code? [y/N] y

conala-paired-train.json: 100% 518k/518k [00:00<00:00, 19.3MB/s]

conala-paired-test.json: 100% 109k/109k [00:00<00:00, 6.90MB/s]

Generating train split: 2379/0 [00:00<00:00, 6772.69 examples/s]

Generating test split: 500/0 [00:00<00:00, 6682.59 examples/s]

Run the cell below to run the training! For the sake of the demo, we just ran it for few steps just to showcase how to use this integration with existing tools on the HF ecosystem.

```

1 dataset

```

```

DatasetDict({
  train: Dataset({
    features: ['question_id', 'intent', 'rewritten_intent', 'snippet'],
    num_rows: 2379
  })
  test: Dataset({
    features: ['question_id', 'intent', 'rewritten_intent', 'snippet'],
    num_rows: 500
  })
})

```

```

1 selected_columns = dataset['train'].select_columns(['intent', 'snippet'])

```

```

1 selected_columns

```

```
Dataset({
  features: ['intent', 'snippet'],
  num_rows: 2379
})
```

```
1 # Assign the eos_token as the pad_token if pad_token is not set
2 if tokenizer.pad_token is None:
3     tokenizer.pad_token = tokenizer.eos_token
4
5 def preprocess_function(examples):
6     inputs = examples['intent']
7     targets = examples['snippet']
8
9     # Tokenize inputs and targets
10    model_inputs = tokenizer(inputs, max_length=512, truncation=True, padding="max_length")
11    labels = tokenizer(targets, max_length=512, truncation=True, padding="max_length")
12
13    # Set the labels as target
14    model_inputs['labels'] = labels['input_ids']
15
16    return model_inputs
17
18 # Apply preprocessing to all dataset splits
19 tokenized_datasets = selected_columns.map(preprocess_function, batched=True)
20
```

```
1 tokenized_datasets
2
```

```
Dataset({
  features: ['intent', 'snippet', 'input_ids', 'attention_mask', 'labels'],
  num_rows: 2379
})
```

```
1 import transformers
2
3
4 tokenizer.pad_token = tokenizer.eos_token
5
6 trainer = transformers.Trainer(
7     model=model,
8     train_dataset=tokenized_datasets,
9     args=transformers.TrainingArguments(
10         per_device_train_batch_size=5,
11         gradient_accumulation_steps=4,
12         warmup_steps=2,
13         max_steps=20,
14         num_train_epochs=5,
15         learning_rate=2e-4,
16         fp16=True,
17         logging_steps=1,
18         output_dir="outputs",
19         optim="adamw_hf"
20     ),
21     data_collator=transformers.DataCollatorForLanguageModeling(tokenizer, mlm=False),
22 )
23
24 model.config.use_cache = False
25
26 trainer.train()
27
```

```

max_steps is given, it will override any value given in num_train_epochs
/usr/local/lib/python3.10/dist-packages/transformers/optimization.py:591: FutureWarning: This implementation of AdamW is deprecated
warnings.warn(
-----
OutOfMemoryError                                Traceback (most recent call last)
<ipython-input-13-d5b17de8c54c> in <cell line: 26>()
    24 model.config.use_cache = False
    25
--> 26 trainer.train()

/usr/local/lib/python3.10/dist-packages/transformers/trainer.py in train(self, resume_from_checkpoint, trial,
ignore_keys_for_eval, **kwargs)
    2120         hf_hub_utils.enable_progressBars()
    2121     else:
-> 2122         return inner_training_loop(
    2123             args=args,
    2124             resume_from_checkpoint=resume_from_checkpoint,
-----
      13 frames
-----
/usr/local/lib/python3.10/dist-packages/transformers/models/gpt_neo/modeling_gpt_neo.py in forward(self, input_ids,
past_key_values, attention_mask, token_type_ids, position_ids, head_mask, inputs_embeds, labels, use_cache,
output_attentions, output_hidden_states, return_dict, cache_position)
    992
    993         # Shift so that tokens < n predict n
-> 994         shift_logits = lm_logits[..., :-1, :].contiguous()
    995         shift_labels = labels[..., 1:].contiguous()
    996         # Flatten the tokens

OutOfMemoryError: CUDA out of memory. Tried to allocate 490.00 MiB. GPU 0 has a total capacity of 14.75 GiB of which 431.06
MiB is free. Process 5633 has 14.32 GiB memory in use. Of the allocated memory 14.03 GiB is allocated by PyTorch, and
171.30 MiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See documentation for Memory Management

```

```

1 model_to_save = trainer.model.module if hasattr(trainer.model, 'module') else trainer.model # Take care of distributed
2 model_to_save.save_pretrained("output")

```

```

1 # lora_config = LoraConfig.from_pretrained('output')
2 # model = get_peft_model(model, lora_config)

```

```

1 # text = "Write a program to sum 2 numbers in python"
2 # device = "cuda:0"
3
4 # inputs = tokenizer(text, return_tensors="pt").to(device)
5 # outputs = model.generate(**inputs, max_new_tokens=200)
6 # print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

```

1 from google.colab import drive
2 drive.mount('/content/drive')

```

Mounted at /content/drive

```

1 Start coding or generate with AI.

```